

HACT: Hyperedge-Aware Certificate Trees for Context-Bounded Multi-Agent Coordination

Research manuscript*

September 18, 2026 | Version 1.0

Abstract

Large agent collectives cannot be coordinated reliably by forwarding every transcript to one model. Bounded hierarchies control individual message size, but a fixed hierarchy ignores which obligations must be rechecked together, and compressed prose does not establish that all required checks passed on the current artifact. We introduce *Hyperedge-Aware Certificate Trees* (HACT), a verification-plane formulation that keeps task ownership separate from evidence aggregation. An edit induces a set-valued invalidation event over an ordered registry of executable obligations. HACT minimizes expected certificate-refresh bytes under a hard per-packet budget by estimating interval activation probabilities and instantiating an exact alphabetic-tree dynamic program. We prove conditional publication soundness, a finite-sample selection bound, and a four-obligation separation showing that identical marginal activity rates can require different optimal trees; any randomized marginal-only selector incurs worst-case additive regret of at least one third of an internal refresh price. In a six-family study with 20 seeds per family, HACT reduces held-out refresh traffic on clustered invalidations by 21.35% relative to a training-selected fixed-fanout baseline and by 21.25% relative to a marginal-only optimizer. The latter comparison reverses on independent invalidations: HACT is 0.51% worse. A separate executable microbenchmark contains 192 mutated Python snapshots and 80 obligations; all five tested layouts agree with a full-test oracle, while HACT reduces serialized refresh traffic by 12.44% relative to the tuned fixed baseline. All certificate packets remain within 3,072 bytes. These are communication and protocol results, not evidence of improved live-LLM reasoning or autonomous repair quality.

Keywords: multi-agent systems; bounded context; executable verification; correlated invalidation; alphabetic trees; reproducible systems research.

1 Introduction

A large multi-agent system has two distinct coordination problems. First, workers must reason about a shared objective while maintaining coherent interfaces and avoiding conflicting writes. Second, the system must determine which outputs are current, which obligations remain unresolved, and whether a release condition has actually been established. Sending all reasoning and all verification details to a central language model conflates these problems. Replacing that model with a fixed tree reduces immediate fan-in, but does not itself determine an efficient evidence topology or preserve the meaning of a completion claim.

Existing research already motivates decentralization. DeLM uses a task queue and shared compact updates to avoid routing every contribution through a main agent [1]. Controlled scaling studies find that the effects of adding agents depend on task structure and coordination architecture [2]. Practitioner guidance similarly advocates selective retrieval and durable notes rather than indiscriminate context accumulation [3]. We do not claim hierarchy, shared memory, selective retrieval, or subagent specialization as inventions.

Our starting point is more specific. Suppose several workers modify a software repository, and a trusted harness maintains a registry of approved checks. An edit to one interface can invalidate its own unit tests, tests of dependent modules, and integration checks simultaneously. The resulting set of stale obligations is

* Author names and institutional affiliations are left for confirmation before submission. The accompanying artifact records AI-assisted mathematical development, implementation, experimentation, and writing.

a *hyperedge*, not a collection of independent scalar activity rates. A certificate tree should place obligations using these joint events, not only a default fanout or individual update counts.

We call the resulting method HACT. The semantic task graph and its write leases remain under the agent runtime. A separate ordered tree aggregates concise, machine-produced certificates. The runtime can change this certificate tree without changing who owns a task, altering an acceptance criterion, or asking an LLM to reread the whole project. At each completed revalidation batch, only activated aggregation nodes emit new bounded packets. A trusted local kernel prevents missing, stale, or incomplete evidence from becoming a publication-level PASS.

Research question. Given a fixed ordered obligation registry, a hard certificate-packet budget, and observations of jointly invalidated obligations, can we reduce refresh communication without weakening the declared verification predicate?

Contributions and attribution boundary. The paper contributes a correlated-activation objective with explicit byte pricing, an exact specialization of alphabetic-tree optimization, an information-loss separation for marginal-only methods, and a freshness-preserving reference kernel that realizes the cost model. It supplies held-out measurements, an executable mutation fixture, adversarial protocol tests, and negative results. The interval dynamic-programming machinery is classical [4]; neither tree optimization in general nor hashing, dependency invalidation, or runtime certificates are claimed as new. The proposed research contribution is the particular problem formulation, its correlation-sensitive consequence, and its validated realization for the verification plane of context-bounded agent systems. Establishing priority against all literature would require a broader review than the targeted search supporting this draft.

2 Related Work and Positioning

Agent coordination and context management. DeLM [1] addresses shared progress and decentralized task claiming. Kim et al. [2] investigate architecture–task interactions, while Anthropic [3] discusses practical context selection. These motivate avoiding an omniscient manager, but they study or guide reasoning organization rather than our ordered, correlated certificate-refresh objective. Our experiments do not reproduce their agent benchmarks, and we make no head-to-head performance claim against them.

Alphabetic and workload-sensitive trees. Vaishnav and Pedram [4] develop generalized alphabetic-tree enumeration and subtree/subforest optimization, including bounded degree and height. Their framework supplies the dynamic-programming template used here; our $O(bn^3)$ complexity is not a new generic tree algorithm. Baer [5] is a further example of optimizing alphabetic codes for nonstandard costs. HACT’s interval weight is the probability that a *set* intersects an interval, not merely the sum of leaf access weights.

The recent Huffman–Merkle Tree work [6] adapts authenticated layouts to changing access frequencies, with batching and tiering. This is a particularly relevant predecessor: workload-aware authenticated trees and amortized layout maintenance already exist. HACT instead studies a fixed leaf order, a byte-constrained aggregation packet, joint obligation invalidation, and a declared gate predicate. Its kernel is not a cryptographic proof system for an untrusted verifier. We compare with a marginal-only optimizer as an information ablation, not with an implementation of HMT.

Incrementality and action certificates. Dependency tracking and memoization are established tools for incremental computation [7]. Proof-Carrying Agent Actions [8] binds governance information to actions across runtimes. These mechanisms address important parts of trustworthy execution. Our incremental dependency closure and publication check reuse these general principles; the optimization target is where complete, current gate evidence should be aggregated when invalidations are correlated.

Table 1: What is inherited and what is investigated here.

Component	Established principle	HACT-specific research object
Agent organization	Hierarchy, shared state, narrow contexts	Separate semantic ownership from the certificate layout
Tree optimizer	Alphabetic interval/forest dynamic programming	Set-activation weights with byte-priced variable arity
Adaptive layout	Workload-aware tree design	Explicit marginal-information separation for co-invalidation
Verification	Trusted checks, dependency invalidation, hash binding	Complete coverage and freshness under tree switching
Evaluation	Held-out comparison and executable tests	Common gate evidence, measured packets, negative controls

3 System Model and the Certificate Interface

3.1 Two graphs, not one omniscient agent

Let the application have an arbitrary task DAG with its own workers, dependencies, and write scopes. HACT does not replace this DAG. Instead, let $\mathcal{R} = (g_0, g_1, \dots, g_{n-1})$ be an ordered, versioned registry of executable obligations. The order may follow modules or an approved contract. An obligation is not necessarily an agent: one worker may discharge many obligations, and one integration obligation may depend on several workers' outputs.

Each g_i declares a predicate, a trusted checker identity, and a complete set D_i of relevant input artifacts. Relevant configuration, checker code, and runtime assumptions must also be bound. A content-addressed artifact snapshot is x . The evidence binding is

$$b_i(x) = \text{Hash}(\mathcal{R}, \text{id}(g_i), \text{checker}_i, \{(a, \text{Hash}(x_a)) : a \in D_i\}). \quad (1)$$

Trusted evidence contains $b_i(x)$, a verdict in $\{\text{PASS}, \text{FAIL}, \text{UNKNOWN}\}$, and a trace commitment. Missing or stale evidence is UNKNOWN. A hash binds records; it does not prove that a check was sound, actually executed, or adequate for the user's objective. Those properties belong to the checker and contract trust boundary.

An artifact change set A invalidates

$$U(A) = \{i : D_i \cap A \neq \emptyset\}. \quad (2)$$

This closure must be complete for the stated predicates. HACT does not infer complete read sets from arbitrary generated code. Undeclared side effects or environment dependence can invalidate the guarantee.

3.2 Ordered certificate trees

A feasible tree has the obligations as leaves in registry order, no unary internal nodes, and between two and b children at each internal node. Each node v covers a contiguous interval $I_v = [\ell_v, r_v]$. Its children form an exact, disjoint partition of that interval. An internal capsule contains the interval, a registry digest, current binding and evidence commitments, and three exact counts: passed, failed, and unknown obligations. It has no model-generated success score.

For children $u_1, \dots, u_k$, aggregation sums the three count vectors and commits to the ordered child records. The verdict is FAIL when any failure exists; otherwise it is UNKNOWN when any obligation is unknown; otherwise it is PASS. Validity additionally requires complete coverage and current bindings. A small count vector is sufficient for the declared conjunction; it is not sufficient to answer every possible semantic question about the project.

The prototype uses a fixed header of H bytes and fixed child cards of S bytes. A refresh packet for arity k has price

$$c(k) = H + Sk, \quad 2 \leq k \leq b = \left\lfloor \frac{C - H}{S} \right\rfloor, \quad (3)$$

where C is the hard packet cap. An oversized mandatory record is rejected, never truncated. Our default $(H, S, C) = (512, 320, 3072)$ permits $b = 8$, but the optimizer can choose smaller arities. The cap concerns this certificate packet, not an entire LLM request containing source files, system instructions, or reasoning history. Any host agent must reserve additional budget for those inputs.

Finite representation. The tested schema supports at most $2^{31} - 1$ obligations and rejects out-of-range or oversized records. This finite bound makes the fixed-width implementation precise. There is no claim that arbitrarily large exact integers fit in constant bytes: an asymptotic encoding requires $\Omega(\log n)$ index/count space.

3.3 Batch semantics and measured cost

A publication claim is revoked immediately when a relevant artifact changes. For communication accounting, an episode is a *completed revalidation batch*: all its submitted check results are incorporated, and each affected internal node refreshes once. Intermediate model conversations, progress polling, and repeated partial refreshes are not counted. A runtime that refreshes on every individual result must record those as separate episodes or use a richer objective.

Let U be the set of obligations whose evidence changes in the batch. An internal node refreshes exactly when $U \cap I_v \neq \emptyset$. Hence

$$L(T, U) = \sum_{v \in \text{int}(T)} c(\deg v) \mathbf{1}\{U \cap I_v \neq \emptyset\}. \quad (4)$$

Gate execution work, source retrieval, and unchanged artifact storage are held fixed across trees. The metric is serialized internal aggregation traffic; it is not network bandwidth measured on a deployed cluster, token billing, end-to-end latency, or total autonomous-agent cost.

4 Hyperedge-Aware Tree Optimization

4.1 Joint activation is the sufficient workload statistic

For a distribution over update sets, define

$$p(i, j) = P\{U \cap [i, j] \neq \emptyset\}. \quad (5)$$

By linearity of expectation, without assuming independent leaf activity,

$$J(T) = EL(T, U) = \sum_{v \in \text{int}(T)} p(\ell_v, r_v) c(\deg v). \quad (6)$$

For m historical episodes $U_1, \dots, U_m$, HACT uses the empirical frequency $\hat{p}(i, j) = m^{-1} \sum_t \mathbf{1}\{U_t \cap [i, j] \neq \emptyset\}$. These interval probabilities compress the workload only for the selected objective. They do not reconstruct the entire hyperedge distribution.

A marginal-only approximation sets

$$\tilde{p}(i, j) = 1 - \prod_{q=i}^j (1 - \hat{p}(q, q)). \quad (7)$$

It uses the same amount of training data, tree family, and optimizer, but imposes within-episode independence. The comparison isolates the value of co-invalidation information rather than an implementation advantage.

Algorithm 1 HACT planning and certificate refresh**Require:** Ordered registry $\mathcal{R}$, trusted checker, historical batches, byte cap C

- 1: Estimate $\hat{p}(i, j)$ from the complete invalidation sets
- 2: Solve Equations (9)–(10); validate exact tree coverage and packet sizes
- 3: **for** each completed revalidation batch **do**
- 4: Revoke publication on the first relevant artifact change
- 5: Compute invalidation closure U from declared dependencies
- 6: Check immutable snapshots; accept only correctly bound trusted records
- 7: Refresh activated tree nodes bottom-up; preserve unchanged valid subtrees
- 8: Compute exact counts and root commitments; retain traces outside packets
- 9: Publish only if the current complete root is PASS under an atomic check
- 10: **end for**

4.2 Exact interval/forest dynamic program

Let $F(i, j)$ be the minimum expected internal traffic for an ordered tree on $[i, j]$. Let $G_k(i, j)$ be the minimum sum of subtree costs for a contiguous forest of exactly k trees covering the same interval. Set $F(i, i) = 0$, and let infeasible states equal $+\infty$. Then

$$G_1(i, j) = F(i, j), \quad (8)$$

$$G_k(i, j) = \min_{i \leq t \leq j-k+1} \{F(i, t) + G_{k-1}(t+1, j)\}, \quad k \geq 2, \quad (9)$$

$$F(i, j) = \min_{2 \leq k \leq \min(b, j-i+1)} \{G_k(i, j) + p(i, j)c(k)\}, \quad i < j. \quad (10)$$

For a fixed interval length, compute its G_k states from shorter intervals, compute F , and then set $G_1 = F$. Record split points and root arity for reconstruction. Using $\hat{p}$ gives the empirical optimum.

Theorem 1 (Exact ordered optimization). *For any valid interval probabilities and price function in Equation (3), Equations (9)–(10) return an optimal feasible ordered certificate tree. The running time is $O(bn^3)$ and retained state is $O(bn^2)$, excluding the workload samples. These are mathematical bounds; the implementation uses float64 arithmetic.*

Proof. Every feasible internal root partitions its interval into $k \in [2, b]$ consecutive nonempty subintervals. Its refresh price depends only on the root interval and k , not on its descendants. Replacing any child by an optimal child with the same interval cannot increase the root term or another child’s cost. Equation (9) enumerates all contiguous k -part partitions by selecting the first child’s right endpoint, and Equation (10) enumerates every allowed root arity. Induction on interval length proves optimality. There are $O(bn^2)$ forest states and at most n candidate split points per state. Each state retains a value and a split; the tree states also retain an arity. Reconstruction adds only linear output size. $\square$

This is an application of generalized alphabetic subtree/subforest optimality [4], not a claim that the recurrence template is new. The interval activation model is the part that changes the agent-coordination problem being solved. Leaf permutations are fixed; the theorem does not imply an optimum over all possible registries or task decompositions.

4.3 Depth constraint and a global-update lower bound

An optimizer minimizing average traffic can create deep branches for rarely observed gates. A hard height budget is an independent control, not a consequence of bounded fanout. Let F_h denote the optimum with at most h root-to-leaf edges. In the forest recurrence for layer h , use F_{h-1} for each child; set singleton costs to zero at every layer. This gives an exact $O(hbn^3)$ extension with retained $O(hbn^2)$ reconstruction state. Feasibility requires $n \leq b^h$. The artifact implements this extension and checks it against exhaustive enumeration. A depth bound limits certificate-chain length, not tool execution or LLM thinking time.

Proposition 2 (Full-refresh cost). *For a tree with n leaves and q internal nodes,*

$$W(T) := L(T, [0, n-1]) = S(n-1) + (H+S)q. \quad (11)$$

Consequently, $W(T) \geq S(n-1) + (H+S)\lceil(n-1)/(b-1)\rceil$.

Proof. The sum of internal arities is the number of edges, $n+q-1$. Substituting into $\sum_v (H+S \deg v)$ gives the identity. Also $n+q-1 \leq bq$, which gives the lower bound on q . $\square$

For $n = 128$ and our packet format, the lower bound is 56,448 bytes. The global-update experiment attains it. Correlation awareness cannot improve a tree already attaining this bound; this supplies an exact negative control rather than a favorable baseline chosen after seeing results.

5 Why Marginal Activity Cannot Determine the Optimum

The main information distinction can be made explicit without a large benchmark. Consider four leaves, binary trees only, and a constant internal-node refresh price $c > 0$. Define

$$\mathcal{D}_A : U = \{0, 1\} \text{ or } \{2, 3\}, \quad \mathcal{D}_B : U = \{0, 3\} \text{ or } \{1, 2\},$$

with probability $1/2$ for each set. Every leaf has activation probability $1/2$ under both distributions. The five ordered full binary tree shapes fall into the cost classes in Table 2. Parentheses indicate binary grouping.

Table 2: An exact four-obligation counterexample. Costs are in units of c .

Tree shape(s)	J_A/c	J_B/c
$((0, 1), (2, 3))$	2	3
$(0, ((1, 2), 3)), ((0, (1, 2)), 3)$	3	2.5
$(0, (1, (2, 3))), (((0, 1), 2), 3)$	2.5	3

Theorem 3 (Marginal-indistinguishability separation). *Any deterministic ordered binary-tree selector that receives only the exact marginal gate activation probabilities has worst-case additive regret at least $c/2$ on $\{\mathcal{D}_A, \mathcal{D}_B\}$. Any randomized selector restricted to the same information has worst-case expected additive regret at least $c/3$. The randomized bound is tight for this two-distribution instance.*

Proof. The root activates surely for both distributions. Each length-three interval also activates surely. A length-two interval contributes its hit probability, giving Table 2. The optimal costs are $2c$ and $2.5c$ respectively. Since the marginal input is identical, a deterministic selector chooses the same shape under both distributions. The three cost classes give regret pairs $(0, c/2)$, $(c, 0)$, and $(c/2, c/2)$, proving the first claim.

For randomization, the last class is weakly dominated by the balanced shape and can be removed without increasing either regret. Let q be the probability of choosing the balanced shape and $1-q$ the probability of an inner-pair comb. Expected regrets are $(1-q)c$ and $qc/2$. Their maximum is minimized where $1-q = q/2$, namely $q = 2/3$, and equals $c/3$. Mixing the indicated shapes with these probabilities attains the value. $\square$

The theorem is not a hardness result for all tree optimization. It identifies information that no marginal-only method can recover, even with an exact optimizer and exact marginal frequencies. A method observing richer co-activity information is outside the lower-bound class. Likewise, the theorem does not establish the necessity of reconstructing a full hypergraph: the interval hit probabilities are sufficient for our objective.

6 Statistical Selection and Freshness Guarantees

6.1 A uniform finite-sample bound

Let $M = n(n + 1)/2$, and suppose the m training episodes are independent and identically distributed. Gate activity within each episode may be arbitrarily dependent. Define

$$\epsilon_m(\delta) = \sqrt{\frac{\log(2M/\delta)}{2m}}. \quad (12)$$

Let $\hat{T}$ minimize the empirical objective over the feasible family, and let T^* minimize its population counterpart. The same family, including any declared height bound, must be used for both.

Theorem 4 (Selection bound). *With probability at least $1 - \delta$,*

$$|\hat{J}(T) - J(T)| \leq W(T)\epsilon_m(\delta) \quad \text{for every feasible } T, \quad (13)$$

$$\begin{aligned} J(\hat{T}) - J(T^*) &\leq [W(\hat{T}) + W(T^*)]\epsilon_m(\delta) \\ &\leq 2(n - 1)C\epsilon_m(\delta). \end{aligned} \quad (14)$$

Proof. For each interval, its observed hit indicators are independent bounded variables across episodes. Applying the two-sided bounded-sum inequality [9], the probability that its empirical frequency differs by more than ϵ is at most $2e^{-2m\epsilon^2}$. A union bound over M intervals gives simultaneous frequency control with the displayed radius. Equation (13) follows by multiplying each interval error by the nonnegative node price and summing. For Equation (14), insert the two empirical objectives and use $\hat{J}(\hat{T}) \leq \hat{J}(T^*)$. A tree with no unary internal nodes has at most $n - 1$ internal nodes, each priced at most C . $\square$

This bound can be loose or vacuous at our sample sizes; it is not used to manufacture an empirical superiority claim. It does establish uniformity over the data-selected tree without union-bounding over every tree shape. It does not cover arbitrary drift or strongly dependent consecutive episodes. In a deployment, trace blocks and retraining policies would need a dependence-aware analysis.

6.2 Conditional soundness of publication

The reference kernel maintains local artifact revisions, scope-bound gate evidence, a complete registry digest, and generation-stamped subtree caches. Its publication ticket identifies a revision and the canonical root record. Relevant changes immediately revoke availability; only fresh checker results restore an affected obligation. A local lock makes comparison with the current snapshot atomic.

Theorem 5 (Declared-contract publication soundness). *Assume that (i) the registry is fixed and includes every obligation in the declared release contract; (ii) each checker is sound for its declared predicate and evaluates immutable inputs; (iii) dependency bindings are complete, their identifiers do not collide on the bound inputs, and relevant changes are recorded; (iv) aggregation verifies exact disjoint coverage and uses only canonical trusted records; and (v) publication compares the current root and revision atomically. Then an accepted root-level PASS implies that every predicate in the declared registry holds on its current bound inputs.*

Proof. A valid leaf PASS corresponds to a current binding and trusted passing evidence, so assumptions (ii)–(iii) establish its predicate. Inductively, an internal PASS has neither failed nor unknown descendants. Exact partition validation prevents omitting a failing leaf, duplicating a passing one, or mixing registries. Therefore its complete interval consists of current passing leaves. At the root this covers all n obligations. Atomic comparison prevents a relevant recorded change between obtaining the root and accepting publication. Any such change revokes the root or changes the revision and rejects the ticket. $\square$

This result is deliberately narrower than “all bugs are fixed.” A finite unit-test registry is not full program correctness, and passing code tests is not evidence that all prose claims are factual. A malicious or mistaken trusted checker violates an assumption. The local lock is not distributed consensus, does not lock arbitrary external files, and does not sandbox generated programs. Missing mandatory details produce an explicit unresolved state, not a guessed success.

6.3 Cache validity must survive topology changes

A subtle implementation failure occurs when validity is tracked only by one global dirty set. Suppose tree T_1 and tree T_2 both have cached roots. An edit invalidates a gate, and refreshing T_1 clears the global dirty set. Reusing T_2 can then revive its pre-edit PASS even though its cached evidence never refreshed.

The implemented correction gives every gate evidence change a monotonically increasing generation. Each cached node stores its topology identity and the maximum generation over its interval. A node is reusable only if that stamp still matches. Since every new generation exceeds all previous ones, any change within the interval strictly increases its maximum. Refreshing one tree cannot make another tree’s stale interval valid. A failing regression trace is preserved in the artifact, followed by the corrected passing test. The mechanism is local cache validation, not a new cryptographic freshness primitive.

7 Implementation and Experimental Protocol

7.1 Prototype and relationship to the agent runtime

The supplied Future Code Control v1.2.0 archive is a Python companion with hierarchical tasks, bounded contexts, durable state, and executable gates. We extracted it, checked its ZIP integrity, and independently reran its existing suite: 263 tests passed. This establishes the available baseline artifact, not an empirical advantage of HACT.

HACT is implemented as a separate research package. Its tree optimizer, certificate kernel, experiments, and tests do not replace the baseline scheduler or claim production integration. A future adapter would consume trusted gate events and dependency closures, then supply bounded certificate packets to existing task coordinators. Model-generated explanations remain separate from canonical verdicts.

The evaluated environment was Linux x86_64, Python 3.13.5, NumPy 2.3.5, SciPy 1.17.0, and Matplotlib 3.10.8. All experiments ran locally on CPU. No hosted language model, paid inference, external deployment, or GPU/NPU was used. Planner timings are wall-clock observations in this container, not dedicated-hardware throughput measurements. The exact environment record is included.

7.2 Common baselines and budget

All methods share the same registry order, invalidation sets, checker outputs, and $(H, S, C) = (512, 320, 3072)$ byte format. Fixed- b baselines build bottom-up consecutive groups of at most b children; singleton groups are promoted without inserting unary nodes. We report fixed arities 2, 4, and 8. A stronger *tuned fixed* baseline chooses one arity from 2 through 8 by minimizing *training* cost, never test cost. The *marginal* baseline uses Equation (7) with the exact same dynamic program as HACT. No baseline is allowed to drop checks, change leaf order, or weaken the context cap.

A flat, all-obligation scanner would require $512 + 320n$ bytes in one packet: 41,472 bytes at $n = 128$. It violates the cap and is therefore not used as a competitive constrained baseline. It remains useful as a conceptual correctness oracle, not as an artificially weak system against which to advertise scaling.

Table 3: Synthetic generators. Gate indices and group ranks start at zero in the implementation.

Family	Definition of one invalidation episode
Singleton	Exactly one gate selected uniformly.
Skewed singleton	Exactly one gate, with probability proportional to $(i + 1)^{-1.1}$.
Independent	Each gate independently included with probability $4/n$; empty sets are retained.
Clustered	Partition into contiguous groups of random size 3–12, truncating the final group. With probability 0.2 choose a uniform singleton; otherwise choose a complete group with rank weight $(r + 1)^{-0.8}$.
Mixed	Use the same group construction. Probability 0.1 selects all gates, 0.3 a uniform singleton, and 0.6 a complete weighted group.
Global	Every episode contains all gates.

7.3 Synthetic invalidation study

The primary protocol was recorded before the main measurements. There are six families, 20 independent seeds per family, and $n = 128$ gates. Each seed has 512 training episodes and 2,048 held-out episodes from independent random streams. In total there are 61,440 training and 245,760 held-out episodes. The retained Boolean masks permit reconstruction of every set.

For each seed and method, the outcome is the mean held-out bytes per episode. Percentage reduction is computed *within each seed* as $100(1 - \bar{L}_{\text{HACT}}/\bar{L}_{\text{baseline}})$ and then averaged across seeds. Confidence intervals are two-sided 95% Student- t intervals over independent seeds. We do not count 2,048 correlated method comparisons within one workload as 2,048 independent replications. Intervals are unadjusted descriptive intervals, not family-wise corrected hypothesis tests.

7.4 Scaling and exploratory sensitivity

The prespecified scaling study uses clustered workloads at $n \in \{32, 64, 128, 256, 512\}$ with four seeds per size. The $n = 128$ values reuse matching seeds from the primary study and are not additional independent trials. The remaining sizes contribute 32,768 held-out episodes. We retain tree depth, maximum packet bytes, refreshed-node counts, full-refresh bytes, and optimizer wall time.

After the main runs, exploratory checks vary training count (32, 128, 512), impose depth 3 or 4, randomly permute the shared leaf order, and evaluate a cluster-trained tree on independent invalidations. These four-seed diagnostics are explicitly post-main-study analyses; they did not select the primary configuration or determine which primary results to report.

7.5 Executable mutation microbenchmark

A separate fixture contains 16 generated Python modules. Module i implements $f_i(x) = x + i$. Four unit obligations evaluate $x \in \{-2, 0, 5, 19\}$. One cross-module obligation checks $f_i(f_{(i+1) \bmod 16}(3)) = 3 + i + ((i + 1) \bmod 16)$. There are therefore 80 obligations in a fixed order.

For each of six seeds, 16 pairs of edits select one to three adjacent modules with a hotspot-weighted starting position. The first edit deliberately adds one to the first selected module’s output; the second restores the intended function. Revision comments ensure that all selected source hashes change. Each seed’s tree is learned from 512 independent training module-edit sets, translated to complete declared invalidation closures. No check result from the 32 evaluation snapshots selects a tree.

Each snapshot is immutable source text. A fresh Python subprocess executes the affected predicates; a separately launched full-registry invocation supplies the oracle comparison. Both use the protected checker, so this is independent *execution scope*, not an independently designed semantic oracle. All five certificate layouts consume the same actual check records. We measure the lengths of packets produced by the kernel and require equality with Equation (4). We also attempt to use the previous publication ticket after each

Table 4: Mean held-out internal refresh traffic in KiB per episode; 20 seeds per family. One KiB is 1,024 bytes. Costs omit tree-invariant gate execution and initial construction.

Workload	Fixed 2	Fixed 4	Fixed 8	Tuned fixed	Marginal	HACT
Singleton	7.88	6.38	7.12	6.38	6.41	6.38
Skewed singleton	7.88	6.38	7.12	6.38	4.83	4.76
Independent	22.23	16.38	17.06	16.08	15.19	15.27
Clustered	12.74	8.80	8.78	8.28	8.27	6.51
Mixed	25.10	15.06	13.20	13.16	14.18	11.92
Global	142.88	74.62	55.12	55.12	55.12	55.12

Table 5: Mean paired percentage reduction by HACT, with 95% seed-level confidence intervals. Positive values favor HACT; negative values indicate a regression.

Workload	Versus tuned fixed (%)	Versus marginal (%)
Singleton	-0.02 [-0.22, 0.18]	0.56 [0.33, 0.80]
Skewed singleton	25.30 [24.81, 25.80]	1.44 [0.93, 1.96]
Independent	5.04 [4.91, 5.17]	-0.51 [-0.63, -0.38]
Clustered	21.35 [20.45, 22.24]	21.25 [20.27, 22.23]
Mixed	9.45 [8.89, 10.01]	15.93 [15.39, 16.47]
Global	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]

edit and require its rejection. The mutations are scripted; no agent is credited with discovering or repairing them.

8 Results

8.1 Held-out traffic and the correlation ablation

Tables 4 and 5 report all six workload families. Figure 1 visualizes paired reductions. On clustered updates, HACT averages 6.51 KiB per episode, versus 8.28 KiB for tuned fixed and 8.27 KiB for marginal-only optimization. The mean paired reductions are 21.35% [20.45, 22.24] and 21.25% [20.27, 22.23], respectively. This supports the proposed co-invalidation mechanism on the tested generator.

The independent family is an important counterexample to a universal advantage. HACT is 0.51% worse than the marginal-only model, with interval [0.38, 0.63]% for the degradation. The factorized model has the correct within-episode structure there, while empirical interval frequencies incur additional estimation variation. HACT still improves over tuned fixed by 5.04%, which distinguishes the benefit of flexible tree shape from the benefit of modeling correlation.

Uniform singleton updates yield essentially a tie with tuned fixed: the paired HACT change is -0.02% $[-0.22, 0.18]$. Skewed singleton updates favor flexible trees over fixed fanout by 25.30%, but HACT’s incremental gain over the marginal optimizer is only 1.44%. The latter comparator incorrectly treats mutually exclusive singleton events as independent Bernoulli activations; it is not an oracle for that family. Mixed local/global episodes produce reductions of 9.45% and 15.93% versus the two stronger baselines. Global updates tie the exact lower bound of Proposition 2. Reporting only the clustered condition would conceal these distinctions.

8.2 Scale, depth, and planner cost

Table 6 shows that the hard certificate-packet cap remains 3,072 bytes through 512 obligations. The planner took 0.008 seconds at 32 obligations and 3.491 seconds at 512 in the measured container. These observations are compatible with the cubic dynamic program; they do not establish a large-cluster deployment limit. At

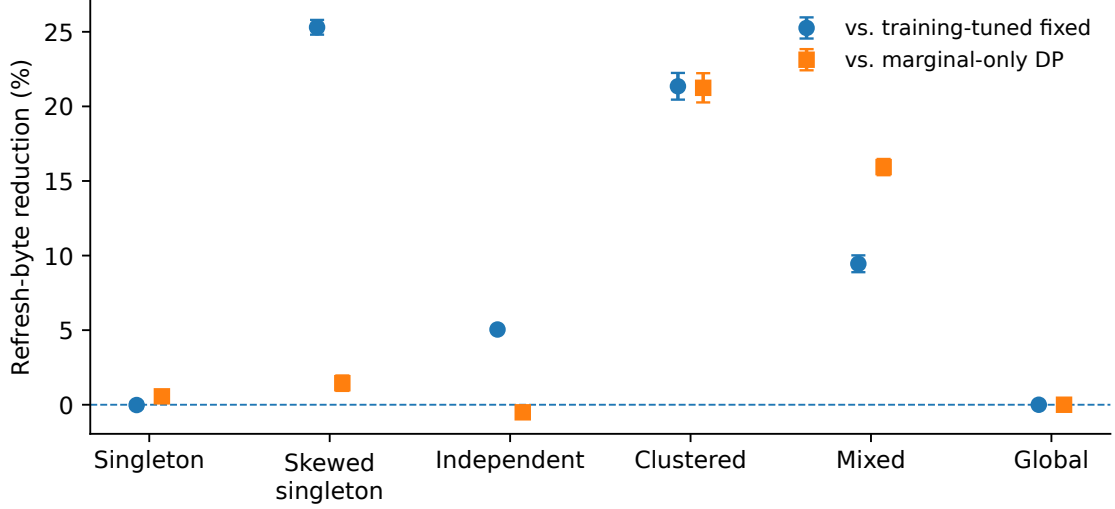


Figure 1: Held-out paired percentage reductions; bars are 95% intervals across 20 independent seeds. The marginal-only comparison reverses under independent invalidations, and the global condition is an exact tie.

Table 6: Clustered scaling study, four seeds per size. Planner times include the optimizer and reconstruction, but exclude workload generation and interval-frequency estimation. Depth is the observed minimum–maximum across seeds.

Gates	Tuned fixed (KiB)	HACT (KiB)	Planner (s)	Depth	Max packet (B)
32	6.04	4.93	0.004	3–3	3072
64	7.24	5.72	0.018	4–4	3072
128	8.33	6.68	0.079	4–5	3072
256	9.40	7.22	0.334	5–6	3072
512	10.60	8.28	1.544	6–11	3072

512 gates the learned depths ranged from 6 to 11, demonstrating why packet size alone is not a latency guarantee.

The gate count is not a live-LLM concurrency count. Internal nodes are deterministic aggregation operations, not persistent managers that each need a language-model session. The prototype remains a single-process local certificate kernel; it does not demonstrate multi-host availability or fault-tolerant distributed scheduling.

8.3 Executable checks and actual serialization

Across the six executable seeds there were 192 evaluated snapshots: 96 faulty and 96 repaired. The selective path ran 1,992 gate checks, while the offline full-registry oracle ran 15,360. Initial certification adds 80 checks per seed and is reported separately from snapshot costs. Selective-check counts are the same for every tree; HACT does not claim their reduction as a topology benefit.

All five layouts agreed with the full oracle on every snapshot. There were zero falsely passing roots among the 480 faulty snapshot–layout decisions, and all 480 repaired decisions passed. All 960 prior-ticket attempts after an edit were rejected. Actual packet lengths agreed exactly with the analytical episode cost, and no packet exceeded the cap. These are finite tests of the declared fixture, not a general estimated failure probability for production.

HACT reduces traffic by a mean paired 12.44% [9.50, 15.38] relative to tuned fixed and 15.99% [10.75, 21.22] relative to marginal-only optimization. Table 7 gives the unpaired method means for context. A safety claim cannot be credited only to HACT here: the sound fixed trees also preserve the predicate. The

Table 7: Executable microbenchmark: actual serialized refresh traffic in KiB per snapshot. Intervals are across six independent seed means. All layouts have identical checker evidence and zero oracle disagreements.

Layout	Mean (KiB)	95% interval (KiB)
fixed4	11.50	[10.81, 12.18]
fixed8	11.12	[10.55, 11.68]
tuned_fixed	10.00	[9.73, 10.27]
marginal	10.44	[9.96, 10.91]
hact	8.76	[8.30, 9.21]

experimentally isolated advantage is lower communication for the same observed gate evidence.

8.4 Exploratory diagnostics and failure modes

For the four diagnostic clustered seeds, using only 32 training episodes produces 8,855 bytes per episode and depths 15–27; 128 episodes gives 6,929 bytes, and 512 gives 6,840 bytes. Sparse empirical data can leave many intervals apparently inactive, leading the unconstrained tie-breaking rule to choose long branches. There is no theorem that a small context automatically induces a good decomposition or adequate training coverage.

A hard depth-three variant averages 6,991 bytes, approximately 2.2% above the 512-example unconstrained result, while reducing all depths to three. Depth four averages 6,836 bytes and happens to be slightly better on held-out data despite a restricted feasible family; empirical optimum need not minimize test risk. These exploratory observations support exposing depth as a separate policy parameter rather than prescribing eight children everywhere.

Randomly permuting the common leaf order raises HACT’s mean traffic from 6,840 to 16,917 bytes; the permuted tuned-fixed baseline averages 21,418. Thus order materially affects absolute cost, even when correlation-aware placement retains a relative advantage on these four seeds. HACT’s optimality theorem cannot be used to claim order-invariant efficiency.

Finally, applying the cluster-trained unconstrained tree to independent test episodes gives 18,007 bytes versus 16,402 for the cluster-selected fixed baseline, a degradation of about 9.8% by the ratio of these diagnostic means. The stationary selection theorem does not protect against this shift. An operational system needs drift monitoring and a conservative fallback; a learned tree should not be frozen indefinitely merely because it was once optimal on training traces.

8.5 Implementation verification

The HACT test suite contains 90 passing cases. It compares the unconstrained optimizer with exhaustive ordered-tree enumeration through seven leaves and the height-constrained optimizer through six leaves. Other cases validate expected-cost identities, the four-leaf separation, disjoint coverage, unknown propagation, stale evidence, tampered displays, publication races, byte limits, and topology-switching freshness. Ten randomized tests exercise 60 edit/revalidation/tree-switch cycles each. A subprocess test independently executes the 80-gate fixture and detects six failed predicates after a single-module fault.

These tests use the actual runtime modules of the research package. The saved failing stale-cache regression demonstrates detection of a genuine implementation error before correction. Passing tests and informal mathematical proofs are not a machine-checked verification of the full implementation. The original companion’s 263 passing tests and HACT’s 90 passing tests are distinct suites, not a combined claim that the new algorithm was already integrated into the original runtime.

An additional retained-data auditor does not import the optimizer, kernel, producer cost helpers, or fixture checker. It recomputes interval traffic directly from Boolean masks, validates tree coverage and source hashes, independently evaluates the fixture arithmetic, and reproduces seed-level summaries. All

primary, scale, and executable records pass this audit; a deliberately corrupted tree is rejected. This is a separate code path, not independent human verification, and exploratory sensitivity records are outside its audit scope.

8.6 Hybrid optimizer and scaling characteristics

The TypeScript port introduces a hybrid optimizer that uses the exact DP for subtrees with at most n_{thresh} gates and a multi-strategy greedy heuristic for larger subtrees. The greedy top level evaluates even splits, p -weighted boundary splits, and local-search refined splits, then selects the split that minimizes exact subtree costs. With $n_{\text{thresh}} = 128$, the hybrid matches the exact DP for all $n \leq 128$ while scaling to $n = 512$.

Table 8 reports byte reductions at $n = 256$ with $n_{\text{thresh}} = 128$, where all six workload families achieve positive reductions versus the balanced baseline.

Table 8: Hybrid optimizer at $n = 256$ with $n_{\text{thresh}} = 128$: mean expected bytes and percentage reduction versus fixed-8 balanced.

Workload	HACT	Balanced	Reduction (%)
Singleton	1597	1664	4.0
Skewed singleton	1184	1664	28.8
Independent	13981	14097	0.8
Clustered	2415	2702	10.6
Mixed	4884	4986	2.1
Global	23424	30336	22.8

9 Deployment Implications, Limitations, and Next Experiments

Keep reasoning distributed, make certification explicit. Workers should retain narrow task contexts, while deterministic software handles ownership, dependency closure, status counts, and publication eligibility. A reasoning coordinator can inspect bounded certificate cards and selectively retrieve implicated traces. It should not invent a global success claim from child prose. HACT supplies only the certificate component of this architecture. Existing context-packet controls must still constrain the rest of each worker’s request.

Changing the tree is not free. Our metric is steady-state refresh traffic. Constructing a new layout requires planning and initializing its internal records; switching may also create scheduling or audit overhead. If a proposed tree is expected to save $d > 0$ bytes per future episode and its migration costs B_{switch} bytes, the simplest break-even count is B_{switch}/d . This arithmetic is not a validated online switching policy. A deployable policy should estimate savings on independent recent batches, account for drift and planning cost in compatible units, and require a margin before replacement. The prototype validates safe reuse when trees switch; it does not implement or evaluate autonomous switch timing.

A small verification context is not a small total intelligence requirement. A conjunction of trusted gate verdicts has an exact compact aggregate. Arbitrary design tradeoffs, theorem discovery, research assessment, and natural-language correctness generally do not. In particular, an encoder supporting exact recovery of every bit of an unrestricted n -bit state without retrieval must distinguish 2^n states and needs at least n bits. Durable evidence and selective reads relocate information; they do not make it disappear. HACT therefore makes no claim that every difficult project can be solved by short independent conversations.

Contract coverage and checker trust. The kernel cannot repair an incomplete acceptance contract, hidden data dependency, compromised checker, or misleading test. Its hashes are not signatures, remote attestation, or proofs of computation. The fixture’s mutations stay within syntactically valid, pure arithmetic modules. Untrusted code needs a genuine sandbox and controlled side effects; a JSON input, subprocess boundary, or local lock alone is not sufficient isolation.

Statistical and external validity. Synthetic groups are deliberately constructed to expose co-invalidation structure; they are not observations of industrial agent traces. The executable fixture is small and highly controlled. Its full oracle shares predicate definitions with the selective checker. Seed-level intervals describe variation under these generators and do not provide confidence intervals for industrial repair quality. Recent agent and authenticated-tree preprints cited here may change. The study does not establish state-of-the-art reasoning performance, production savings, or superiority over full implementations of DeLM or HMT.

Required real-agent evaluation. The next substantive test should freeze real repository tasks and gate registries, collect invalidation episodes from actual model-generated edits, and compare fixed, marginal, and HACT certificate layouts under the same model, prompts, tool permissions, seeds, and aggregate budget. Required endpoints include verified task success, false publication attempts, request tokens, repeated evidence retrieval, end-to-end latency, migration cost, and traffic. These measurements would determine whether certificate savings are material relative to inference and tool costs. They have not been executed in this manuscript and are not filled in with synthetic agent scores.

10 Conclusion

The relevant alternative to one large managerial context is not simply a larger hierarchy. It is a separation between distributed reasoning and explicit, bounded verification. HACT models repeated gate invalidation as correlated set activity and chooses an ordered certificate tree under a packet budget. Its core optimization is an attributable specialization of classical alphabetic dynamic programming; its information separation explains why marginal activity can be insufficient. The trusted kernel preserves declared coverage and freshness independently of the chosen tree, including after topology changes.

Executed experiments support a limited but concrete conclusion: for the tested correlated workloads, HACT reduces certificate-refresh traffic without changing the observed verification predicate. Independent and global controls, sparse-data depth problems, and a drift regression delimit that benefit. A live-model study remains necessary before extending these results to autonomous software-engineering quality or total agent-system efficiency.

Reproducibility and Disclosure

The accompanying research artifact contains the source optimizer and kernel, 90 tests, the frozen primary experiment contract, all main and scaling workload masks, per-episode measurements, all executable snapshot sources and checker records, aggregation code, the $\text{\LaTeX}$ manuscript, and file hashes. The supplied v1.2 baseline ZIP is retained separately for provenance; it is not silently patched. Synthetic and executable measurements were produced during this work, not borrowed from cited papers or from the earlier illustrative agent demos. AI assistance was used for research, mathematical development, implementation, testing, and writing. Human authors should review the proofs, results, attribution, and authorship before submission. No peer review, formal proof-assistant validation, or institutional endorsement is asserted.

References

- [1] Yuzhen Mao and Azalia Mirhoseini. Decentralized Multi-Agent Systems with Shared Context. arXiv:2606.10662v1, June 2026. <https://arxiv.org/abs/2606.10662v1>.
- [2] Yubin Kim et al. Towards a Science of Scaling Agent Systems. arXiv:2512.08296v3, revised April 8, 2026. <https://arxiv.org/abs/2512.08296v3>.
- [3] Anthropic. Effective Context Engineering for AI Agents. Engineering article, September 29, 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- [4] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; publication date not established from the retrieved copy. <https://www.mpedram.com/Papers/alp-trees-journal.pdf>.
- [5] Michael B. Baer. Alphabetic Coding with Exponential Costs. arXiv:cs/0605099v3, revised March 2009. <https://arxiv.org/abs/cs/0605099>.
- [6] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026. <https://arxiv.org/abs/2608.25206v1>.
- [7] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006. <https://doi.org/10.1016/j.entcs.2005.11.043>.
- [8] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104, June 2026. <https://arxiv.org/abs/2606.04104>.
- [9] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. <https://doi.org/10.1080/01621459.1963.10500830>.

A Exact Data Generation and Reproduction

The synthetic generator uses NumPy’s `default_rng` initialized with a `SeedSequence` whose entries are `[260918, n, seed, stream]`. Stream zero produces training episodes and stream one held-out episodes. Group construction uses `SeedSequence([7001, seed, n])`. Each group draws an integer size from 3 through 12 and truncates at the final leaf. Pilot seeds 1000 and 1001 are excluded from all main tables. Scaling sizes other than 128 have their own saved records; the four reused 128-gate seeds are clearly identified in the analysis.

The executable fixture uses `SeedSequence([83012, seed, stream])`. It contains one fault and one repair per selected module group; the two phases within a pair are not treated as independent replications. Interrupted fixture runs append completed snapshot records to a journal and replay those records when resuming. They do not reclassify incomplete work as a completed seed. The summary is emitted only after all 32 snapshots for that seed exist. Runtime interruptions encountered during development motivated this resumable execution path; final tables use complete records only.

The main commands, from the extracted research root, are:

```
python -m pytest -q
python -m experiments.synthetic --family clustered --n 128 --seeds 20
python -m experiments.executable --seeds 6 --out results/executable_fresh
```

```
python -m experiments.sensitivity
python -m experiments.aggregate
python -m experiments.validate_results
```

The fresh executable output directory prevents replay of the supplied journal. Copying new results into the canonical results directory before aggregation is an explicit operator action; the README details that step. The README gives all six primary-family commands, the scaling loop, bounded resumable fixture commands, and the $\text{\LaTeX}$ build. The artifact separates raw results from regenerated tables. Nothing in these commands contacts an LLM endpoint.

B Certificate Encoding and Trust-Boundary Details

Each capsule contains `range`, `counts`, `binding`, `evidence`, and `registry`. Hash fields are 64-character lowercase SHA-256 hexadecimal encodings. Canonical JSON uses sorted keys, ASCII escapes, and compact separators. In a packet, the header occupies 512 bytes including its newline; each child card occupies 320 bytes including its newline. Padding makes byte accounting exact. Fixed padding is not asserted to be a token-optimal production encoding. A binary or tokenizer-aware encoding would define a new price function and require re-evaluation.

The trusted verifier API rejects mismatched bindings, invalid verdicts, and malformed trace hashes. It is intentionally not exposed as a tool through which an untrusted LLM can certify its own work. Returned display capsules are compared with canonical trusted records rather than accepted because their self-reported hashes appear valid. This local design demonstrates record integrity and freshness under its trust assumptions; distributed authenticated transport and credential isolation remain host-runtime responsibilities.

The optimizer’s $O(bn^3)$ claim concerns planning. The reference kernel prioritizes clarity and scans interval generation stamps; it also hashes tree descriptions. We do not infer an asymptotically optimal runtime-update implementation from the communication bound. Large-scale implementations could use range-maximum indexes and cached topology digests, but those optimizations are outside the evaluated prototype.

C Evidence-to-Claim Register

Table 9: Audit map for the paper’s main claims.

Claim	Evidence	Scope boundary
Exact ordered optimum	Theorem 1; exhaustive enumeration tests	Fixed order and price model
Marginals can be insufficient	Theorem 3; exact five-shape test	Specified binary instance
Finite-sample selection control	Theorem 4	IID stationary episodes; bound may be loose
Declared-contract soundness	Theorem 5; stale and tamper tests	Trusted complete checker contract
Clustered traffic reduction	20 held-out workload seeds; raw masks and costs	Synthetic workload distribution
Executable traffic reduction	Six seeds, 192 snapshots, actual packets	Small scripted mutation fixture
Packet cap maintained	Serializer checks, scale records, packet-length assertions	Certificate packet, not full model request
Production or LLM-quality improvement	Not measured	No such claim